

Convoze

App Name

Convoze

App Description

Convoze is a real-time 1:1 chat app built with a Flutter mobile client and a Node.js/Socket.IO backend on PostgreSQL. It uses phone number + OTP authentication (no passwords, no email) and a local-first client architecture — the app reads only from an on-device database that stays in sync with the server through a per-user "Update log," so messaging keeps working offline and catches up instantly on reconnect.

App Summary

Convoze is being built from scratch as a documented, from-first-principles messaging app, with domain terms pinned down in **CONTEXT.md** (Session, Presence, Read/Delivery watermark, Update log, Sync cursor, Local replica, Outbox, etc.). The backend is server-trusted by design (deliberately not end-to-end encrypted) so it can offer server-side search, moderation, and multi-device sync without the constraints E2EE would impose. The client is Flutter + Riverpod + Drift (SQLite), talking to an Express + Socket.IO + Prisma/Postgres backend.

Tech Stack

Mobile (**mobile/**)

- Flutter (Dart), Riverpod (**flutter_riverpod**, code-gen via **riverpod_generator**)
- **go_router** for navigation
- **drift** + **sqlite3_flutter_libs** for the on-device local replica (local-first data layer)
- **dio** for HTTP, with an auth interceptor for token refresh
- **socket_io_client** for the realtime connection
- **flutter_secure_storage** for JWT storage
- **image_picker**, **file_picker**, **flutter_contacts**, **permission_handler** for media and contacts

Backend (**backend/**)

- Node.js (22.18+), Express 5
- Socket.IO for the realtime transport (WebSocket-only, no long-polling)
- PostgreSQL 18+ with Prisma ORM
- Twilio Verify for OTP delivery/verification
- Cloudinary for media storage, transforms, and signed uploads
- **jsonwebtoken** for access/refresh tokens

- **libphonenumber-js** for phone number parsing/normalization
- Tests: Node's built-in test runner, **supertest**, real **socket.io-client** connections against a real Postgres test database

Security Used

- **Phone + OTP auth, no passwords:** Twilio Verify handles OTP generation/verification server-side; the backend never stores an OTP or a password.
- **Enumeration-safe endpoints:** **POST /auth/otp/request** responds identically whether or not the number is registered; 404s are collapsed so a non-participant can't distinguish "doesn't exist" from "not yours to see."
- **Rate limiting:** OTP requests (3 per phone number per 15 minutes via an event-log table), contact sync/lookup (30 per 10 minutes), media upload signatures (60 per 10 minutes), and message sending all have independent limits.
- **JWT access tokens (15 min TTL)** carrying a **sessionId**, checked against a server-side **Session** row on every request — so revocation (logout, logout-others, refresh-token reuse) takes effect immediately instead of waiting out the token's expiry.
- **Refresh token rotation with reuse detection:** refresh tokens rotate in place on every use (30-day sliding TTL); reusing an already-rotated or revoked token is treated as a theft signal and revokes that session outright.
- **Multi-device session model:** each device has its own revocable **Session**; a revoked session's live sockets are disconnected immediately server-side.
- **Secure token storage on-device** via **flutter_secure_storage**, with single-flight token refresh on 401s.
- **Signed, validated media uploads:** uploads go straight to Cloudinary via short-lived signed parameters; the server validates type/size and never trusts client-supplied URLs.
- **TLS in transit.** Convoze is explicitly server-trusted, not end-to-end encrypted — this is a deliberate, documented tradeoff to keep server-side search, moderation, and multi-device sync, not an oversight.

Features

(Only features actually implemented in the codebase — backend route + client UI — are listed.)

- **Phone number + OTP sign-in**, with per-device sessions and silent token refresh
- **Real-time 1:1 messaging** over Socket.IO, with an offline-first outbox (composed messages persist locally until acknowledged, including failed sends)
- **Message replies** (reply to an earlier message; the preview reflects the original's live state, not a frozen snapshot)
- **Message editing and deletion**
- **Emoji reactions** on messages
- **Media sharing:** images and files via Cloudinary, client-side picking/compression

- **Read and delivery watermarks** ("delivered"/"seen," derived per-conversation rather than stored per message)
- **Presence:** online status derived from live socket connections, with a persisted "last seen"
- **Conversation preferences:** pin (max 5), archive, mute (8h / 1 week / always), clear chat history, delete chat — synced per-device for the owning user only
- **Contacts:** match phone contacts against registered users, find a user by phone number
- **User profiles:** display name, about/status text, avatar
- **Conversation search**
- **Local-first client:** on-device SQLite replica (Drift) that the UI reads from exclusively, kept in sync via a per-user Update log and sync cursor so the app works offline and resumes cleanly
- **Test-only e2e harness** (fault injection, database reset/seeding, token TTL overrides) for reliable end-to-end testing of the above

Future Scope

Documented in [docs/features.md](#) as designed-but-not-yet-built:

- Push notifications (FCM/APNs) for offline devices — pipeline designed, not yet implemented
- Link previews (client-generated Open Graph cards)
- Polls (up to 10 options, visible votes, in group chats)
- Voice notes, @mentions, message requests
- Voice and video calling
- Blocking / reporting users
- Deliberately deferred: view-once media, chat folders, on-device (client-side) search, local database encryption, content-free push, Stories, and end-to-end encryption (a documented non-goal, not just "not yet")

USP / How Convoze Differs from WhatsApp

- **Documented-first build.** Every non-trivial decision (session model, sync protocol, media pipeline) is written up and justified before implementation — the project favors a documented, defensible architecture over shipping fast and patching later.
- **Explicit, honest security posture.** Rather than claiming privacy it doesn't have, Convoze states plainly that it is server-trusted (TLS only, not E2EE) so it can support server-side search and moderation — a tradeoff WhatsApp doesn't need to make explicit since it markets E2EE as default.
- **Update-log sync model.** Instead of ad hoc "did I miss anything" reconciliation, every durable change is a positioned entry in a per-user Update log with a client-held sync cursor — a Kafka-esque, replay-based approach to catching a device up after any amount of offline time.
- **References, not snapshots.** Replies and read/delivery status always resolve to the *current* state of the thing they point at (an edited or deleted original shows correctly in a reply), rather than freezing a copy at creation time the way Signal's quote-reply does.

- **Outbox-first composition.** Messages a user has sent live in an explicit, inspectable Outbox — including failed ones — instead of silently disappearing or being retried invisibly.
- **Architectural transparency over scale.** Convoze is a portfolio/learning build, not a production competitor to WhatsApp or Signal — it doesn't have their scale, audit history, or years of hardening yet. What it offers instead is a sync protocol, session/device model, and every security tradeoff written down and justified rather than left as opaque implementation details.